NexusAI - Admin Configuration & Project Specifications

A guide to system architecture, dynamic settings, and knowledge ingestion metrics.

1. Project Overview

NexusAI (formerly DevPilot AI) is an advanced agentic software engineering and research assistant platform. It orchestrates autonomous workflows using custom large language model prompts, dynamic execution verification, and runtime isolation. The goal of the system is to allow users to build complete software projects, run competitor analyses, interact with tutorials, and execute automated tasks seamlessly.

2. System Workspaces & Modules

- **Engineer AI:** Supports autonomous project code generation, interactive review sessions, build testing, and repository pushing.
- **Research AI:** Runs detailed competitor research, market analysis, compiles timeline details, and generates download-ready markdown reports.
- **Education AI:** Interactive programming workspace for custom tutorials, database guides, and concept tutoring.
- **Automation AI:** Execution and scheduling workspace for backend flow integrations and workflows.
- **Dynamic MCP Registry:** A registry allowing runtime local (stdio) and remote (SSE) Model Context Protocol tools to interface with AI context.

3. Suggested Admin Data & Knowledge Inputs

To customize NexusAI's agent behavior and context, admins can safely ingest the following metadata/knowledge files into the agent's context base (RAG/memory databases). Do not include raw credentials or connection secrets in these datasets.

Knowledge Category	Suggested Data to Include	Usage / Context Value
Coding Standards	Language styling guidelines, naming conventions, lint rules.	Ensures generated code aligns with team standards.
Workspace Policies	Allowed workspaces, directory structures, build file definitions.	Keeps agent tasks confined to correct directories.
MCP Integrations	Commonly approved local processes (e.g. database schemas, filesystem limits).	Helps agent choose correct servers for task execution.
Architecture Standards	Preferred technology stacks (FastAPI, React, SQLite, Tailwind version).	Prevents compilation issues on final build outputs.

4. Data Security Policy (Safe vs. Sensitive Data)

When populating system prompts or RAG knowledge bases, adhere to the following safety specifications:

- **Safe Data (Shareable):** API endpoint signatures, technology choices, allowed paths, directory maps, schema names, and deployment templates.
- **Sensitive Data (PROHIBITED):** Database passwords, secret tokens, private SSH keys, personal access tokens (PATs), and raw session tokens. All secrets should be injected via environment variables (.env files) rather than static documentation.

5. Main Tech Stack Specifications

- **Frontend:** Vite + React + CSS (Glassmorphism layout + Responsive design).
- **Backend:** FastAPI, Uvicorn, Python driver processes.
- **Database / Storage:** MongoDB for active collections (e.g. dynamic MCP servers), ChromaDB for vector memory indexes.
- **LLM Layer:** Groq, dynamic JSON-RPC MCP clients for stdio/SSE.